



软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



第六章 面向对象的软件架构设计

主要内容

- 6.1 概述
- 6.2 设计类接口和操作
- 6.3 设计信息隐藏类
- 6.4 设计继承
- 6.5 类接口规约
- 6.6 类的详细设计



6.1 概述

- 信息隐藏

- 设计者需要确定哪些信息应该隐藏在类中，哪些信息应该放在类的接口中

- 接口与实现分离

- 接口是接口的提供者和接口的使用者之间的一种契约

- 顺序性面向对象软件架构设计

- 拥有一个控制线程
- 对象间的通信采用调用/返回这一种通信模式



6.2 设计类接口和操作

- 类接口由每个类提供的操作组成
 - 输入参数
 - 输出参数
 - 返回值
- 类的操作可以根据静态模型或动态模型确定
 - 静态模型用于展示类的操作
 - 动态模型用于确定类的操作



6.2 设计类接口和操作

- 基于交互模型设计类的操作
 - 当两个对象交互时，发送消息的对象调用接收消息的操作
 - 消息被映射为一个操作调用
 - 消息的名字映射为操作的名称
 - 消息的参数映射为操作的参数

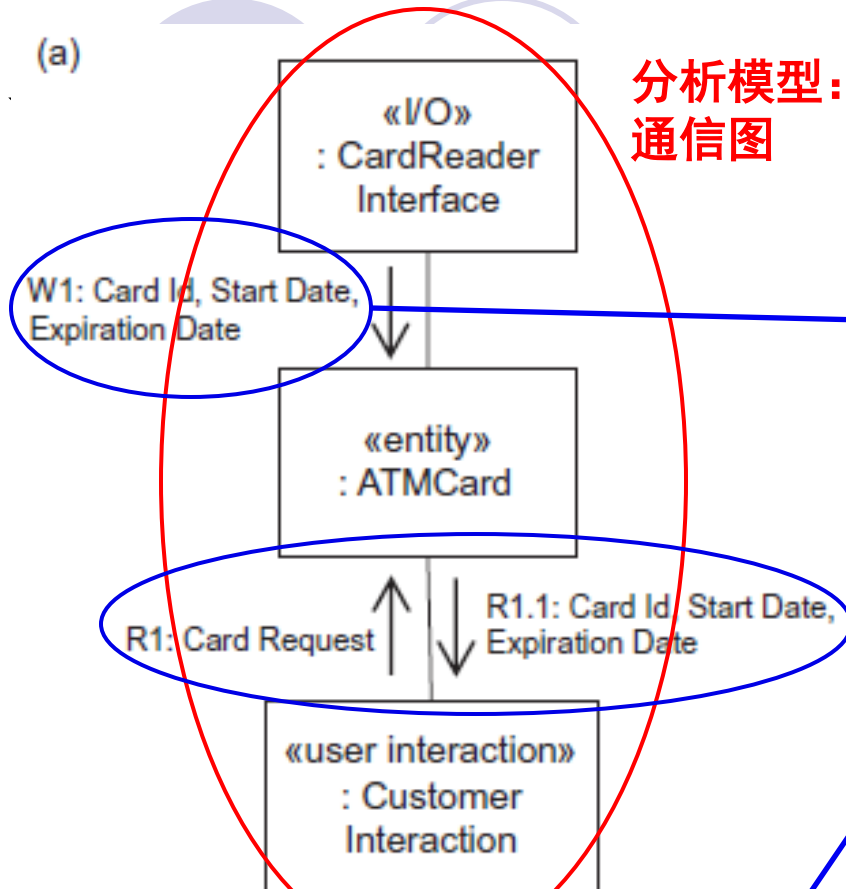


6.2 设计类接口和操作

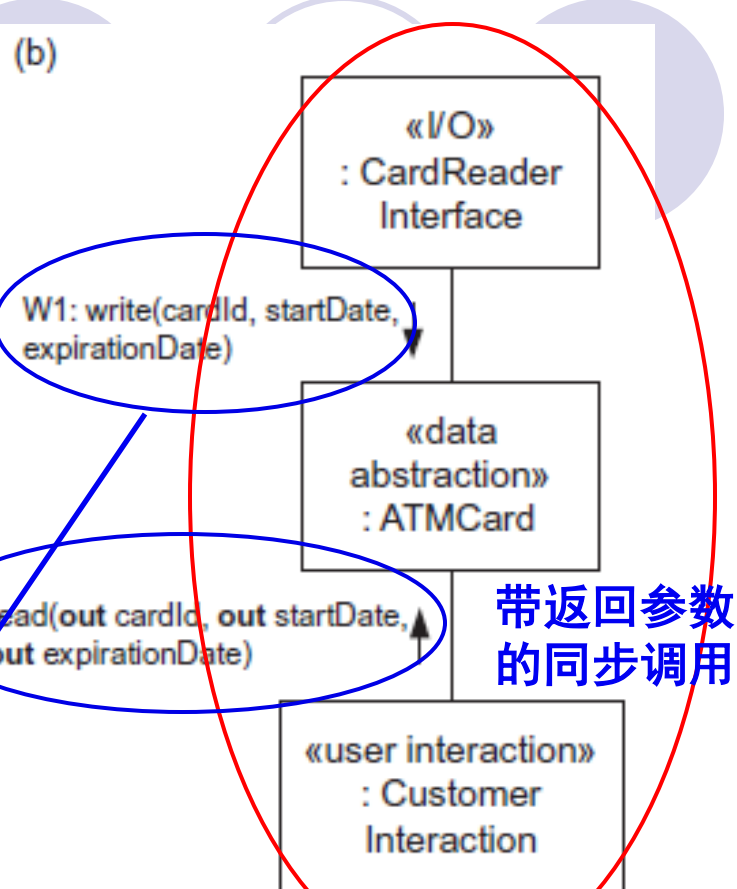
- 分析模型捕捉对象间传递的信息，而不是操作的准确语法，通信图上的消息可能是所传递的数据（用名词表示），也可能是要执行的操作（用动词表示）
- 在设计模型中，若消息为名词，则需定义接收消息中数据的对象的操作；若消息为动词，则该动词表示操作的名称
- 调用操作的消息在通信图中被描述为同步消息
- 表示应答的简单消息被映射为操作的返回参数



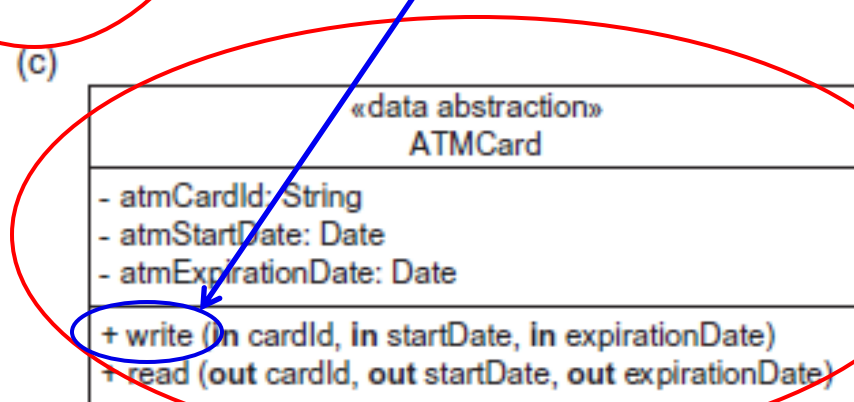
(a)



(b)



(c)

设计模型：
通信图设计模型：
类图

西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

6.3 设计信息隐藏类

● 实体类

- 是封装数据的类，其实例是用来存储信息的持久化对象
- 实体类分成 **数据抽象类** 和 **包装器类**
 - 数据抽象类：封装数据结构
 - 包装器类：隐藏了访问存储在现有系统或遗产系统中的数据的访问接口



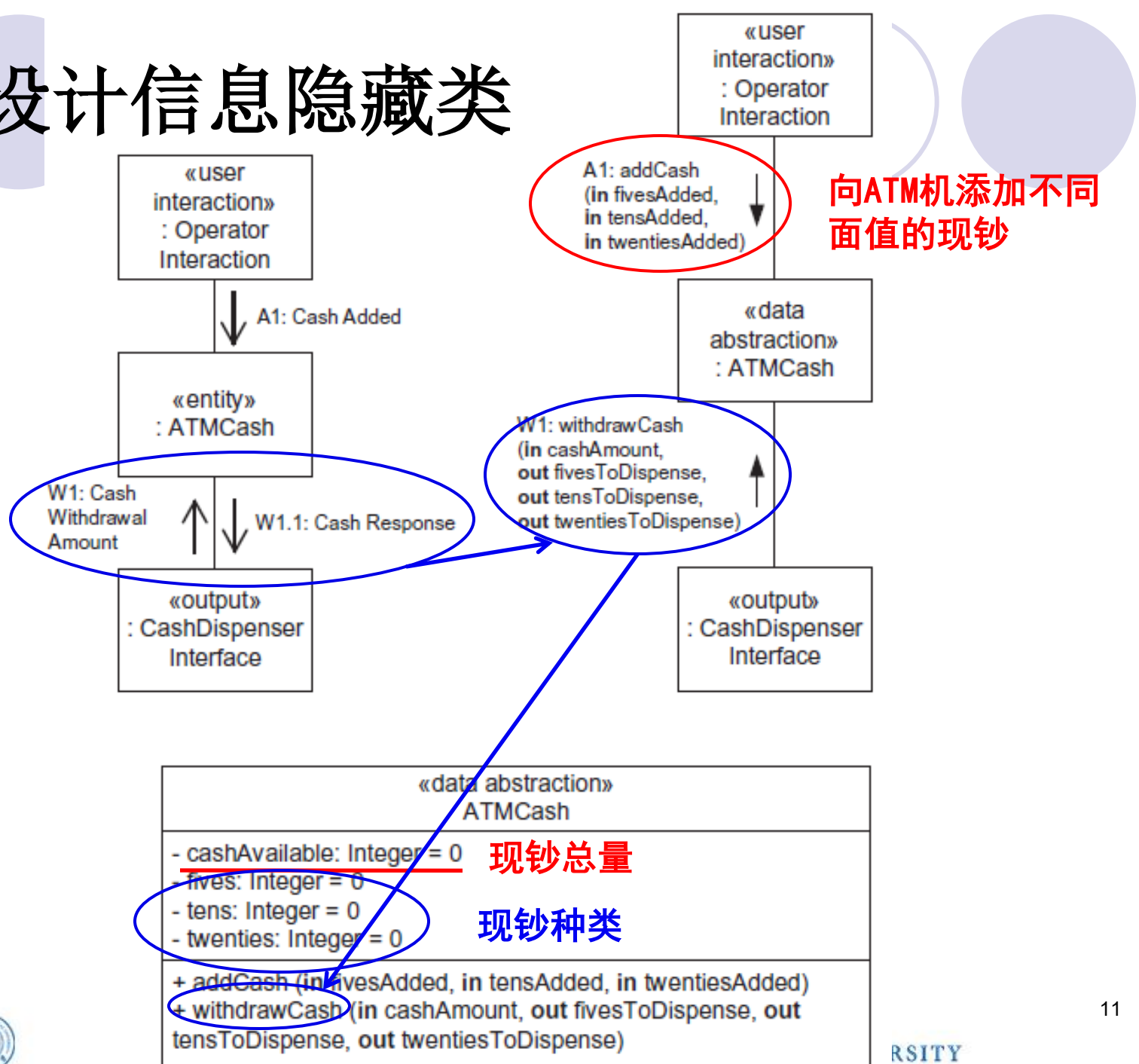
6.3 设计信息隐藏类

○ 数据抽象类的设计

- 分析模型中的实体类被设计为 **数据抽象类**
- 封装了数据结构并隐藏了数据结构的内部细节
- 属性信息在实体类静态建模时已经确定
- 操作被设计为访问过程或访问函数，其内部定义的关于如何操纵数据结构的细节也被隐藏
- 通过分析其他对象访问数据抽象对象的方式来确定操作
- 例子：“ATMCash” 现金类



6.3 设计信息隐藏类



6.3 设计信息隐藏类

● 控制类

- 为一组对象提供总体协调
- 被动控制类： **状态机类**
- 主动控制类： 协调者类和计时器类



6.3 设计信息隐藏类

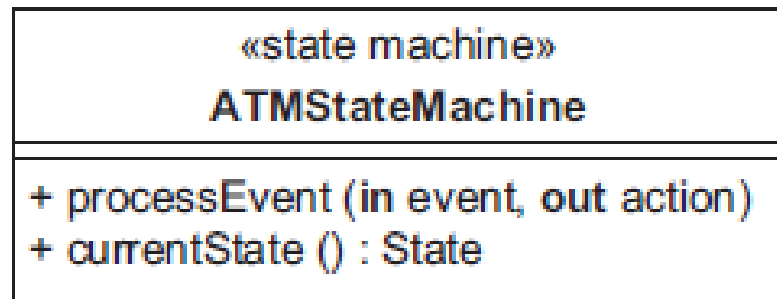
○ 状态机类的设计

- 封装了一个状态图所包含的信息
- 状态机对象所执行的状态图用状态转换表描述
- 状态机类隐藏了状态转换表的内容，并维护对象当前状态
- 状态机类提供了访问状态转换表和改变对象状态的操作
- 一般会设计一个或多个操作来处理导致状态改变的输入事件，操作和事件一一对应，但这种设计无法复用
- 可以设计可复用的状态机类



6.3 设计信息隐藏类

- 可复用的状态机类提供了两个通用的可复用的操作
 - **processEvent(in event, out action)** //当有新事件需要处理时被调用，它查找状态转换表来确定此事件的影响，考虑状态机的当前状态和必须满足的条件，新状态是什么及需执行的动作；接着改变当前状态，返回需要执行的动作作为输出参数
 - **currentState(): State** //返回状态机当前状态
- 状态机类可复用，但封装的状态转换表的内容依赖于特定应用
- 状态机类被实例化或初始化时定义 *状态转换表的内容* 以及 *初始状态值*



6.3 设计信息隐藏类

- 边界类

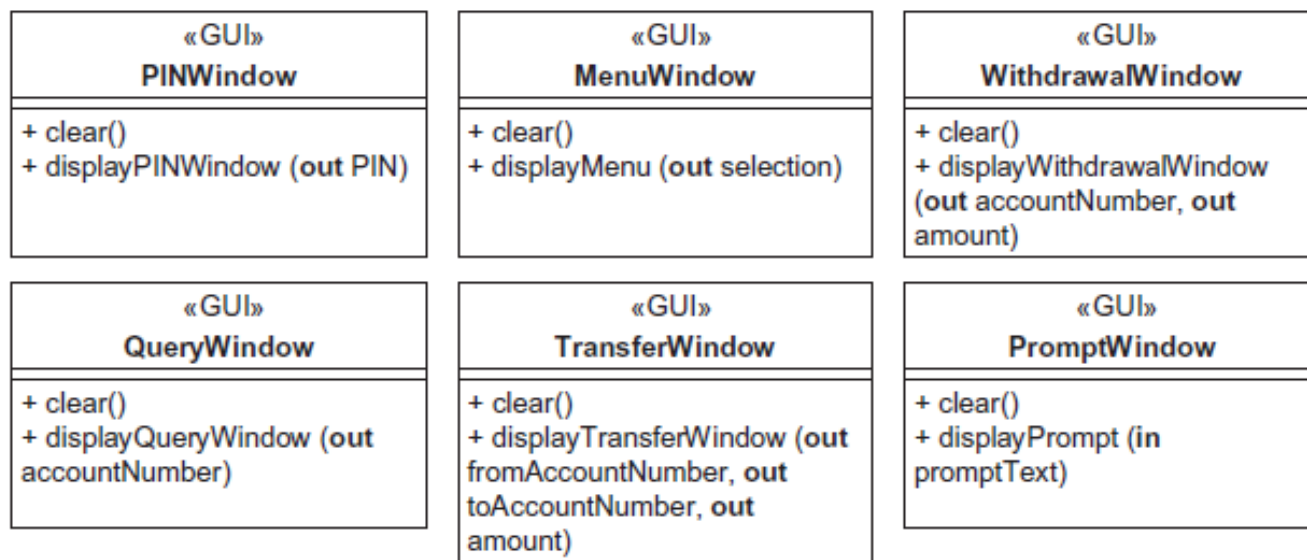
- 与外部环境通信
- 通常是主动类：I/O类、代理类
- 也可以是被动类：**图形用户交互类**
 - 为用户提供访问接口并为它们呈现信息



6.3 设计信息隐藏类

○ 图形用户交互类的设计

- 该类向其他类隐藏了与用户界面相关的细节
- 用户界面可以是简单的命令行或**GUI**
- 在分析模型中，用户交互类捕捉用户输入的信息以及向用户显示的信息
- 而每一个**GUI**屏幕画面所需要的**GUI**类都需要设计



6.3 设计信息隐藏类

- 应用逻辑类

- 封装特定应用的逻辑和算法
- 主动类：算法类通常是主动的
- 被动类：**业务逻辑类**和服务类



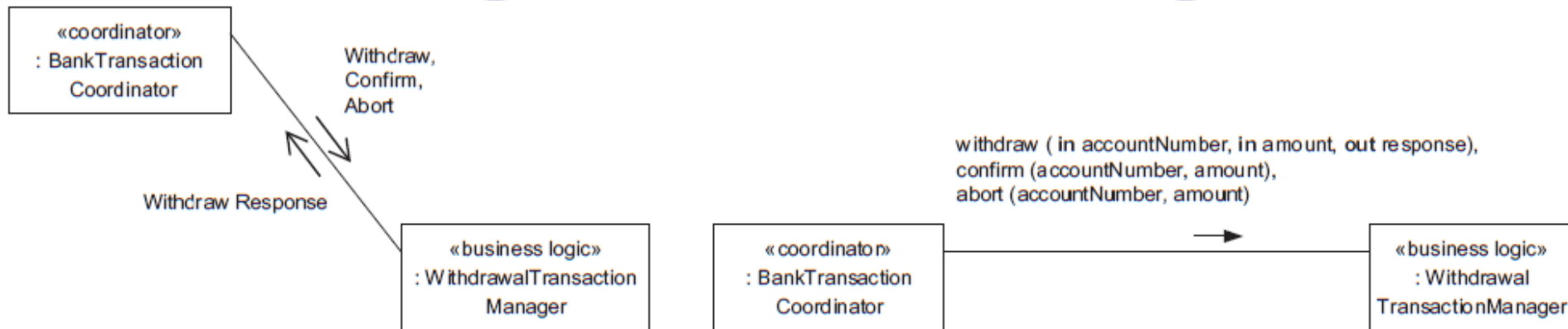
6.3 设计信息隐藏类

○ 业务逻辑类的设计

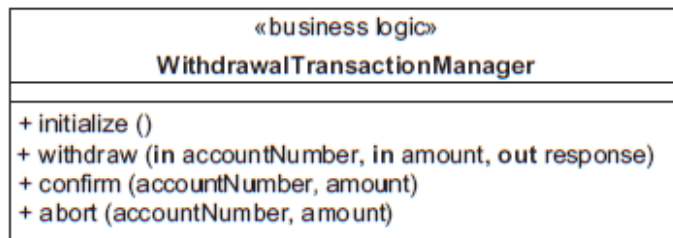
- 定义了处理客户端请求的决策以及特定业务的应用逻辑
- 目的是将可能会相互独立变化的业务规则封装到不同的业务逻辑类中
- 业务逻辑对象通常会在执行过程中访问多个不同的实体对象
- 例子：“取款交易管理器”——封装了处理**ATM**取款请求的规则



6.3 设计信息隐藏类



(c)



6.4 设计继承关系

● 类继承

○ 自顶向下的设计方法

- 先设计刻画一组类的总体特征的超类
- 再设计子类之间的差异性

○ 自底向上

- 初始设计包含具有一些共同特性及一些不同特性的一组类
- 再将共同特性泛化为一个超类
- 超类包含的属性和/或操作被不同的子类共同继承

○ 混合方法

限制类继承
层次的深度
以避免波纹
效应



6.4 设计继承关系

● 抽象类

- 不能实例化，可以被用作创建子类
- 抽象类只能用来作为一个超类并且为其子类定义公共接口
- **抽象操作**在抽象类中声明但没有实现，一个抽象类至少包含一个抽象操作
- 若抽象类被具体类继承，则所有抽象操作必须在子类中实现，否则子类也是抽象类
- 抽象类也可以实现一些操作，这些操作被它的部分或全部子类所共享，但子类可以**重写**这些操作

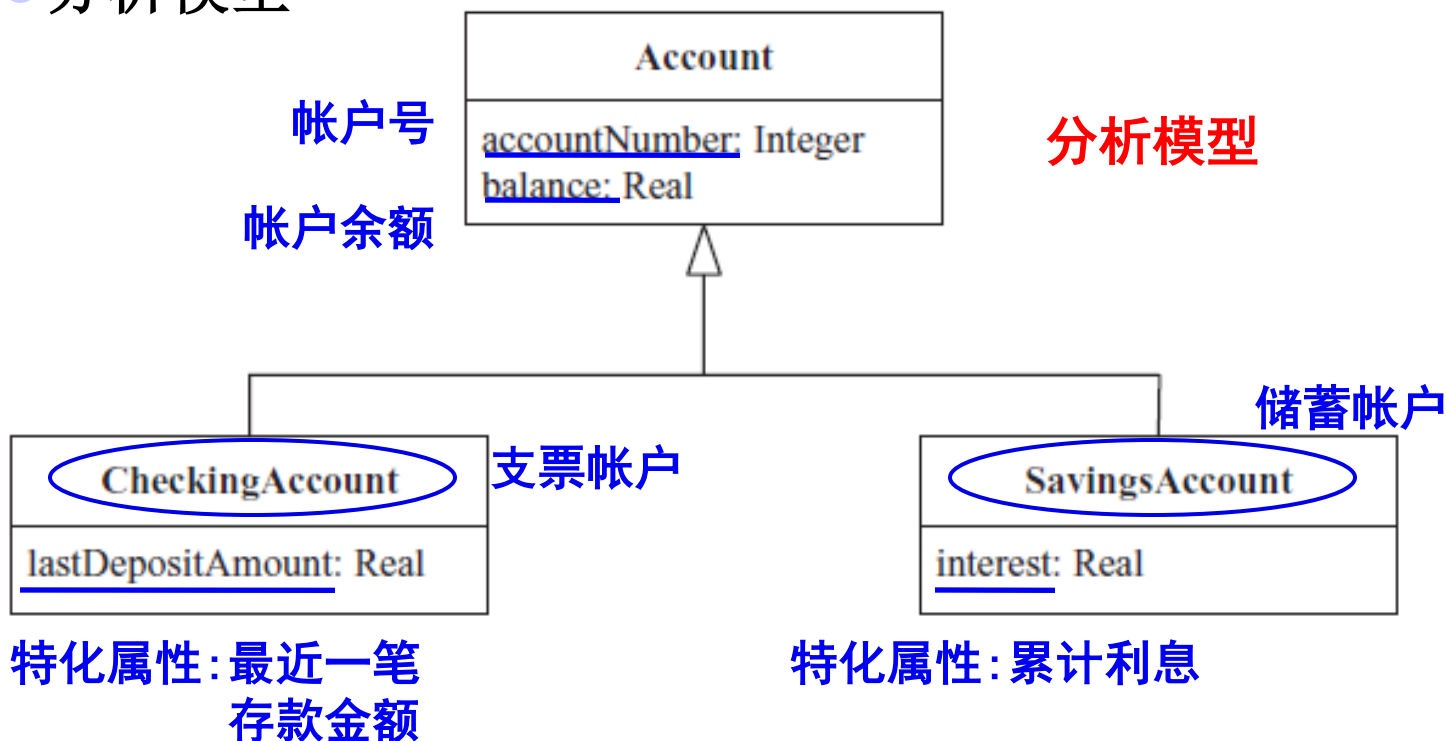
抽象类是一种
通过继承来共
享特征和行为
的机制



6.4 设计继承关系

○ 抽象类继承的例子：账户

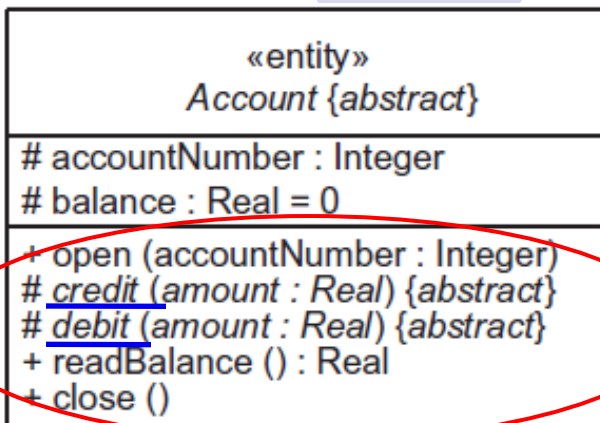
● 分析模型



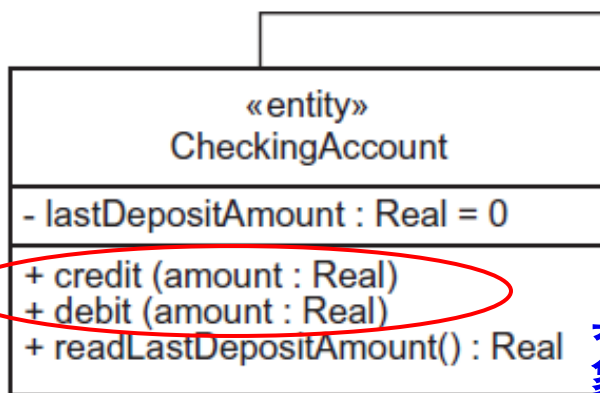
6.4 设计继承关系

● 设计模型

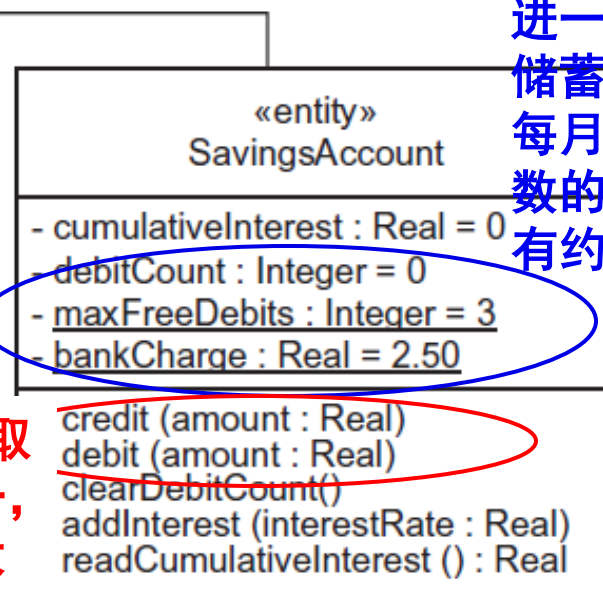
由于各种账户存款和取款的处理方式不同,所以在超类中用抽象操作,在子类中具体化为不同实现



对所有账户都应有的泛化操作



具体实现抽象操作



进一步设计:
储蓄账户对每月取款次数的手续费有约定

取款:从账户余额中减去取款金额

取款:从账户余额中减去取款金额,将取款次数加一,如果取款已经超出了最大免费取款次数,则从账户余额中扣减一笔手续费



西北工大

NORTHWESTERN POLYTECHNICAL UNIVERSITY

6.5 类接口规约

- 定义了信息隐藏类的 **接口**
 - 信息隐藏类所隐藏的信息
 - 类组织准则：类所属的类别
 - 定义类时所作的假设
 - 可能的变化
 - 超类（若有的话）
 - 继承的操作（若有的话）
 - 该类所提供的操作



6.5 类接口规约

○ 操作的定义:

- 所执行的功能
- 前置条件
- 后置条件
- 不变量（在任何时刻都必须成立的条件）
- 输入参数
- 输出参数
- 所使用的其它类的操作



6.6 类的详细设计

- 在信息隐藏类详细设计过程中，重点是确定每个操作的内部算法
 - 算法用伪代码形式描述
 - 独立于特定编程语言
- 例子：支票账户子类的详细设计



■ Attributes:

- Inherit: accountNumber, balance 继承的属性
- Declare: lastDepositAmount 自己定义的属性

■ Operations:

- Inherit specification and implementation: open, close, readBalance
- Inherit specification and define implementation:

继承的规约及其实现

credit(**in** amount : Real); 继承规约, 但自己
定义其实现

begin;

Add amount to balance;

Set lastDepositAmount equal to amount;

end.

- Inherit specification and define implementation of:

debit (**in** amount : Real);

begin;

Deduct amount from balance;

end.

- Add operation:

readLastDepositAmount () : Real

begin;

return value of lastDepositAmount;

end.

